🗺️ 0.0 Navigation

Handshake AI

Project Phoenix

Instructions Document

0.0 - Navigation, Welcome, and Context 🗺️

This is a living document. Last Updated 10/25/25

This page is a central navigation hub for all essential Project Phoenix resources. Below, you will find clickable links to the two main sections of the Project Phoenix instructions documents, along with a brief description of their contents. Welcome, Phoenixes! See the playbooks for more specific tab navigation.

Document Section	Description
📕 **1.0 Fellow Playbook** **New Fellows - Start here!**	This section provides an introduction to Project Phoenix, outlines the covered math sub-domains, and details the steps to get started with tasking, including practice and full-time project onboarding. This section also includes how to write strong tasks, task checklists, golden task examples, fellow FAQs, and much much more. Dive in!
📕 **2.0 Reviewer Playbook** **New Reviewers - Start here**	This section provides an introduction to reviewing on Project Phoenix. This section includes how to evaluate tasks, rubrics, reviewer FAQs, how to evaluate model responses, reviewer pod resources, and much much more!

1.0 Fellow Playbook

Handshake AI

Project Phoenix

Fellow Playbook

1.0 - Navigation

This is a living document. Last Updated 11/13/25

Main Menu

Click any title to go to the corresponding tab

Document Tab	*Contents*
1.1 Onboarding	• Onboarding and Step-by-Step • Fellow Expectations • Helpful References
1.2 Tasking	• Overview and Goals of Tasking • Technical instructions • How to write strong tasks • How to respond to reviews
1.3 Tips and Tricks	• Helpful tips for tasking
1.4 Golden Task Examples	• Learn from video examples what high-quality tasks look like and how they were conceived
1.5 How to Write "Research Level"/Niche Tasks	• Some mathematicians are deep in a niche subfield that other reviewers may be unfamiliar with. Please read this guide to produce in-depth tasks while also making it easy on your reviewers!
1.6 Fellow FAQs	• Frequently asked questions by fellows

Welcome to the Project Phoenix Fellow Team! If you've been contacted directly via email, you've been selected based on your Hanshake Profile and strong fit for the project. Please read the tabs in this document (or click the links in the menu above) to navigate through your journey as a fellow to submit frequent, high-quality tasks.

Goals of the project

Submit high quality original proof statements and solutions to the Handshake platform.

Fellows: Your Role

- Create original, high-quality proof statements and solution-based tasks that fail 2/4 AI models in the practice project, and 2/4 AI models in the production project.

Benefits

- Timely feedback and support on your reviews and tasks
- Direct coaching from STEM experts in office hours
- Future opportunities to grow with the Handshake AI team as a star fellow or reviewer

Expectations

- Complete onboarding
- Maintain an approval rate above 30% once you pass practice (practice tasks do not count towards or against your approval rate)
- Maintain consistent quality and timeliness in responding to reviews

👋 1.1 Onboarding

This is a living document. Last Updated 11/12/25

Menu

Click any title to go to the corresponding section

Section	*Topics*
Overview, Goals & Step by Step	• Math Subfields • Onboarding step by step
Expectations	• Fellow Performance Standards • Paths to Achievement + Bonuses • Review timeline
Helpful References	• OH Calendar & Weekly Sessions • Pronto • Contact Support

Overview, Goals & Step by Step

Welcome to Project Phoenix! In this project, your goal is to create original, graduate-level mathematical prompts that cause AI models to fail at proof-writing. Below we will help you through your fellow onboarding process when you have been added to the project.

Covered Math Sub-Domains

Here is a list of some of the sub-domains of mathematics you can choose from in the Handshake AI Annotations platform. Your sub-domain of expertise is likely covered here and super valuable for
our project - and these are just a few of the options! If your field is not listed here, you are welcome
to write tasks for your field, and choose the “closest relative” to that field.

Analysis

- Real Analysis
- Complex Analysis
- Functional Analysis
- Measure Theory
- Ordinary Differential Equations (ODEs)
- Partial Differential Equations (PDEs)

Algebra

- Group Theory
- Commutative Algebra
- Noncommutative Algebra
- Homological Algebra

Number Theory

- Algebraic Number Theory
- Analytic Number Theory
- Combinatronics

Geometry

- Algebraic Geometry
- Differential Geometry

Topology

- General (Point-Set) Topology
- Algebraic Topology
- Differential Topology
- Low-Dimensional Topology

Logic & Foundations

- Set Theory
- Mathematical Logic
- Category Theory

Probability & Statistics

Other

- We accept any field - just label “other”

Onboarding Step by Step

1. **If you are new to our project, you will start on the "Project Phoenix (Production)" project.**
    - Go to domain model training and look through the Slack Materials
    - Once you read the training, please begin tasking.
    - For your first task, you can submit only one task at a time.
    - Our Reviewers will carefully look through your attempt. If it meets our acceptance criteria (see tasking for our guidance on quality and acceptance criteria!), we will approve your task, where you can begin unthrottled tasking.
    - If your task is approved, you will receive an email
    - If your task is rejected, you may **not** receive an email. Make sure you check your "my tasks" to see its status.
        - "Not started" = the task is ready for you to work on or address reviewer comments
        - "Review 1" = a reviewer will complete your review soon
        - "Ready to deliver" = Your task has been approved!
        - "Delivered" = Your task has been approved!

2. **You will receive a $150 bonus in your first paycheck when your practice task is approved.**
    - You will then be automatically onboarded to our production project and will sign a Deel contract. This enables us to pay you the onboarding bonus + your ongoing hourly compensation of $75/hour.
    - We will not be able to pay your onboarding bonus if you do not follow the steps and sign a Deel contract.
        - Note: You must have US citizenship or have CPT/OPT to receive a contract in deel. Make sure your profile information is set to "US" and a US state is selected.

3. **Once you reach Project Phoenix, you can begin unlimited tasking!**
    - You will also become eligible for bonuses and perks like exclusive Handshake AI swag for continuing your journey with us.

Expectations

Fellow Standards

In Project Phoenix, we have the following standards for our fellows in the production project.
Once you have completed practice, we will begin tracking progress and approval rates (submissions/acceptances). We also have checks for LLM similarity, plagiarism, and user inactivity.

Approval rates <30%
We will offboard fellows if their approval rate is less than 30% over their 5 most recent unique tasks (meaning across 5 tasks, they have had 17 or more attempts total). Note that the practice task does not count against the approval rate.

Plaigiarism
If we detect plagiarism (copied work from other fellows, examples, previous tasks in Handshake, or verbatim copied from the internet, etc.), we will offboard on a case-by-case basis, depending on the severity.

LLM use
If we detect LLM use in proofs, we will offboard fellows on a case-by-case basis, depending on the severity. We will send warnings if we detect LLM use.

Inactivity
If fellows are inactive for more than two weeks, we will offboard them and assume they are no longer available to participate in the project.

Paths to Achievement + Bonuses (DEPRECATED)

In Project Phoenix, we have the following achievement and bonus structures for our fellows in the production project.

Leveling Leader Board (total hours on project)
On Canvas here, you can find the total hours on project leaderboards. As you reach each benchmark on the leaderboard, you can earn special

Ember: 30 - 100 paid work hours
💵 $100 when you hit 30 hours
🎖️ Credly badge + certificate
🎁 Handshake AI swag 📰 Access to Fellow Newsletter

Blaze: 100 - 160 paid work hours

💵 $250 when you hit 100 hours
🥇 Credly badge + certificate
👕 Exclusive Project Phoenix swag
📰 Fellow newsletter feature opportunity
🎓 Invites to AI Research Talks

Inferno: 160+ paid work hours
💵 $500 when you hit 160 hours
🥇 Credly badge + certificate
🎒 Elite Project Phoenix merch
📣 Company-wide shout-out
🌟 VIP experience, more details to come! (In the past, this has entailed a visit to our SF office!)

Promotions to Star Fellows
We will promote fellows to "Star fellows" if they have an approval rate of 80% or higher across their 5 most recent tasks (meaning across 5 tasks, they have had 6 or fewer attempts total).

Bonuses Structures
Star fellows are eligible for weekly bonus compensation structures on top of their hourly pay. Bonuses range from $100 - $400

Promotions to Reviewers
We will promote high-quality fellows to the role of reviewers. Specifically, we will focus on strong
mathematical content and lower overall production capacity.

Review Time

Practice Task Review Priority
We will always prioritize our new fellows in the practice project to ensure they receive feedback early in their process!

Average review time is ~1 day
Despite our efforts to get to every new task the same day it is submitted, our average review time has been around 1 day. When you make it to production, we recommend submitting 1-2 tasks a day so you have a constant stream of new tasks and can review feedback on older tasks to work on.

We are doing our best & thank you for your patience
Please keep in mind that we are all one team here on Project Phoenix, and we want to get to your tasks and changes as soon as we can! Thank you so much for your patience and understanding as we may take longer to get to some tasks than others.

Helpful References

Slack Calendar

For our weekly and ongoing events, please also refer to Slack for the Calendar of sessions, Zoom links, and recent announcements. Here is an overview of our weekly session.

Office hours: Come with any questions you may have, or just come and get to know your project lead and reviewer team!

Math task-a-thon: Sharpen your tasking skills and get live, actionable feedback! How it works:

1. Pitch: Bring a new task idea, something the model might struggle with.
2. Proof: Show your reasoning or example failure.
3. Polish: Get feedback, refine, and submit.
4. Rewards: Top task wins Handshake AI swag, and everyone leaves with clear next steps.
5. Bring: A task statement + solution pair, plus a brief rationale for why it's interesting.
6. Tips: Aim for clarity, novelty, and keep your pitch concise (~3–5 minutes).

Task Excellence Webinar: Come to see your reviewers and experts walk through some new golden tasks, and give you pro tips for writing high-quality tasks!

Typical Weekly Schedule

Tuesdays: Office hours 2:00 - 2:30 pm PST
Thursdays: Office hours 2:00 - 2:30 pm PST

Slack

Slack is a space for group discussions and peer-to-peer help. You may notice fewer interactions from the Handshake team here. If you need urgent help, please reach out to Fellow support (see section below).

Handshake AI Support

If you have any issues, whether technical, financial, or accessibility related, please contact our support team at fellow-support@joinhandshake.com. They will be happy to help you with your problem. For effective requests, please provide as much detail as you can.

✍️ 1.2 Tasking

This is a living document. Last Updated 11/11/25

Menu

Click any title to go to the corresponding section

Section	*Topics*
Overview & Goals	• Welcome and general principles
Technical Instructions	• Tasking Step-by-Step Process

Writing Tasks	• How to Write Great Prompts
	  ○ [Checklist]
	• How to Determine if Models Failed
	  ○ [Checklist]
	• How to Write Great Proofs
	  ○ [Checklist]

Responding to Reviews	• What to keep in mind when receiving reviews

Overview & Goals

Welcome to Project Phoenix! In this project, your goal is to create original, graduate-level mathematical prompts that cause AI models to fail at proof-writing. Below are some key things to keep in mind when tasking on this project.

> **This project is focused on teaching AI models to write high-quality proofs, so there are high standards for your prompts and proof statements.**

- Your prompts and proof statements do not just have to be correct – they also have to be clear, fully justified, and well-written, with proper KaTeX formatting.
- Reviewers will be entering your prompt and proof statement on the client's annotation platform, and must be able to copy and paste your prompt and proof statement. (They can only fix minor typos or formatting issues that don't affect meaning.)
- For these reasons, you may feel that your tasks are getting sent back for more minor errors, compared to other projects.

We need prompts to be original, graduate-level math questions that require a proof to solve and that stump the models.

- AI labs already have access to online resources, so a goal of this project is to generate original problems that can't be found elsewhere.
- AI models have been getting better and better at solving difficult proof-based questions. To ensure that our tasks push the boundaries of what the models know, it is crucial that prompts induce model failures, regardless of whether you think the problem is difficult or not.
- Since this project is focused on teaching AI models how to write proofs, prompts must actually require the model to produce a proof.

Do what you can to get your tasks approved, and please do not argue with reviewers.

- We are all on the same team, and reviewers are trying to approve as many tasks as possible while still ensuring they meet the high standards set by the client.
- If you get feedback from a reviewer, lean toward implementing it unless you believe it is incorrect or violates the project guidelines. Remember that reviewers will be submitting your task and grading models on the client's annotation platform, so in addition to ensuring your proof is high-quality, they also need to feel comfortable with your solution.
- If you feel that a reviewer is exhibiting bad behavior or not following the project guidelines, you can submit a dispute form here: **[Dispute form link]**

Technical Instructions

Step-by-Step Process

1. **Navigate to the Handshake annotation platform and claim a task.**
    - [Annotation Platform Link]
    - Once your practice task is accepted, you will claim tasks under "Project Phoenix."

2. **Write an original, graduate-level mathematics prompt that must be answered with a proof.**
    - Your prompt must be an original, clearly written, unambiguous, graduate-level mathematics question or statement that requires a proof to answer (and uses proper KaTeX).
    - As a rule of thumb, tasks should be accessible enough that a generalist math PhD can understand and review them within an hour.
    - See below for guidelines on writing prompts, and see this guide to KaTeX formatting.

3. **Review model responses and select which model(s) failed.**
    - For your practice task, at least two of the models must fail. For regular (production) tasks, at least two of the models must fail.
    - See below for information on what counts as a model failure.

4. **Write a proof that is clear and fully answers the prompt.**
    - Your proof must be understandable by a generalist math PhD, must be fully justified, and must be clearly written using KaTeX formatting.
    - We recommend working in a separate document, then pasting your work into our platform.
    - See below for guidelines on writing proofs.

5. **Provide any necessary sources for your prompt or proof, and select the sub-field of mathematics that your prompt falls under.**
    - If your specific subdomain is not listed, select the option on the list that is most similar to the subdomain of the problem.

6. **Write a brief description of the model failures.**
    - Your answer here helps reviewers check that your prompt induced valid model failures, so that they don't have to spend time rereading the model proofs themselves.
    - Make sure your answer highlights the major model failure(s) and briefly explains why the model(s) are wrong.

7. **If your task gets returned to you by a reviewer, make necessary changes and resubmit the task.**
    - Carefully read all reviewer comments and make any changes that are necessary. Respond to reviewer comments about major or minor errors, and indicate whether the issue has been resolved, then resubmit your task.

- You can check the status of your tasks at any time using the annotations platform:
    - "Not started" = the task is ready for you to work on or address reviewer comments
    - "Review 1" = a reviewer will complete your review soon
    - "Ready to deliver" = Your task has been approved!
    - "Delivered" = Your task has been approved!

Writing Tasks

How to Write Great Prompts

Goals:

Ensure your prompt is original.

- Prompts can be inspired by existing sources, but cannot be copied word-for-word and cannot just involve language changes. For example, rephrasing 'Is November a rainy month in London?' as 'Is it rainy in the capital of England in the month following October?' does not make it a new question.
- See the sources for task inspiration below the rubric!

Ensure your prompt requires graduate-level math knowledge and asks for a proof.

- Prompts need to explicitly ask for proof (so the models actually generate proofs), but they do not have to use "prove or disprove" specifically. Other ways to ask for a proof include "prove that," or "demonstrate/show with proof."

Ensure your prompt is clearly written and unambiguous.

- If models or reviewers are confused by your prompt, it will be sent back.
- You should include definitions or clarifications of terms that are ambiguous (e.g., does "manifolds" include "manifolds with boundary"?) or not well known.

Ensure you only include one proof question in each task

- Each task must include **only one** proof statement. Prompts with multiple proof requests will be returned for edits.

Avoid:

Do not submit multiple versions of the same task with only minor details changed.

- If you find that a certain prompt stumps the models easily, it can be tempting to submit additional tasks using similar prompts. However, this violates our requirement from the client that problems be original.
- Once you have submitted two similar tasks, any other similar tasks will be rejected, and you will be asked to change your prompt.
- Submitting overly or purely computational proof statements

Checklist for Prompt Writing	
Requirement	**Detail**
Prompt must be original.	Your prompt should not appear verbatim in any research paper, textbook, or online forum (e.g., Math Stack Exchange). Your prompt should also not be logically similar to other problems. Do not submit statements or proofs that only involve changes in language. If the underlying statement or logic remains the same, we will return the tasks to you for revision. Minor wording changes to well-known problems (e.g., classical theorems or common textbook exercises) do not count as original.
Prompt must be clear.	Your prompt should be clearly written, with all assumptions stated, and there should be no ambiguity in the definitions you are using. If the models or reviewers do not fully understand the prompt, your task will be rejected.
Prompt must be well-written.	Your prompt should be written with correct spelling, proper grammar, and proper KaTeX formatting. See this guide (on the Phoenix Canvas page) for advice on writing in KaTeX.
Prompt must require graduate-level math and a proof.	Your prompt must involve graduate-level math and require a proof to solve. While you don't need to use the specific phrase "prove or disprove," your prompt should ask the model for a proof. (For example, instead of "calculate this integral," you can say "determine, with proof, the value of this integral.)
Prompt must not be a trick question.	Your prompt should not ask the models to prove something that is false, or try to confuse the model in ways not related to mathematics. For example: do not ask the model to prove a false statement, and do not ask the model to "prove or disprove" whether [object X] has [property A] when X does not exist.

Prompt must fall within the field of mathematics.	Your prompt should be a mathematics problem. If your prompt is inspired by another field, like physics or finance, make sure you provide all necessary background information and state all relevant definitions in purely mathematical terms.
Prompt must not be computationally intensive	Your proof can involve use of a calculator, but prompts which require more advanced computational techniques to solve (e.g., python scripts) are not allowed. Moreover, we do not count model failures which are due to calculation errors.

Good sources of task inspiration:

🔍 **Research Papers**

1. arXiv
2. ResearchGate

📚 **Textbooks**

1. Graduate Texts in Mathematics (GTM)
2. Graduate Studies in Mathematics (GSM)
3. Cambridge Studies in Advanced Mathematics

🌐 **Online Sources**

1. Math Stack Exchange
2. Math Overflow
3. The Stacks Project
4. What's new

How to Determine if Models Failed

Goals:

Ensure that the models made significant errors in reasoning or proof-writing.

- Do not submit a task if the models failed for only minor or subjective reasons. For example, if the models fail to read an image correctly, make a minor arithmetic error that doesn't impact their proof, or use an approach that isn't how you would solve the problem (but is still correct), this does not count as a failure.
- See the rubric below for more information on what counts as a valid model failure.

Be as clear as possible when explaining the major model failure(s).

- Make sure your answer points out specific parts of the model response(s) and explains why they are wrong.

Avoid:

Do not ignore the first part of the model's response; this is what you should focus on.

- Some of the models give a proof and then provide a behind-the-scenes "reasoning" section at the end. In general, you can ignore the bottom "Reasoning" section and only focus on the first part of the model response, which comes before this.

What Counts as a Model Stump?		
Model Error	**Valid Stump** ✅ **or** **Invalid Stump** ❌	**Reason**
The model's citations, knowledge recall, or references are wrong.	✅	This stump is acceptable. We do not want the models to reference incorrect or false sources, and we do not want the models to use false information to justify steps in a proof.
The model misreads an image.	❌	We are looking for reasoning errors that occur after correct interpretation of the problem. We want the failure to be logic-based and not due to misreading or being unable to process an image.
The model makes small computational errors (as long as the proof is still correct overall).	❌	Sometimes the models make small typos within the proof that they do not use in the remainder of the proof. In this case, there is no true logical error. For example: if the model incorrectly uses $a-b$ in one line, but in the remainder of the proof it uses the correct quantity $a+b$, this does not count as a model stump.
Ambiguous definitions: You specify the definition used in the statement/prompt, but the model uses an incorrect definition in its proof.	✅	If the prompt and associated definitions are clearly provided and the model still uses the incorrect definition, this is a valid model failure.
Ambiguous definitions: You DO NOT specify the definition used in the statement/prompt, and the model uses an incorrect definition in proof.	❌	We want the model to be stumped by a logical issue. We don't want stumps that occur due to ambiguity in the prompt. Some common examples: - "Manifolds" may not include "manifolds with boundary". - "n-differentiable functions" standing for C^n-functions.

The model fails due to a lack of prompt clarity.	❌	Again, we want to ensure that the model fails due to logic and reasoning error and not due to an ambiguous or unclear prompt.
The final answer is correct, but there is not enough detail in the proof.	Gray area	We want the model to achieve a good balance between sufficient detail without overexplanation. Egregious lack of detail is a problem, but in general, it is okay for the model to use commonly accepted statements with little justification. We understand that this can be a gray area, and ultimately, reviewers will be the ones to decide this.
Rambling: Model says something incorrect first, then corrects itself.	❌	The end result of the model reasoning is most important. As long as there is a valid proof at the end of the model's response, this is not a valid model stump.
Rambling: Model says something correct first, then changes its answer to something incorrect.	✅	The end result of the model reasoning is most important.
The model gives a strong counterexample that could work, but the explanation is incorrect or incomplete.	✅	Because we are concerned with proper justification and not just the correct conclusion, a valid counterexample without appropriate justification is considered a valid stump.
The proof's final answer is correct, but the proof uses incorrect logic.	✅	This is an acceptable failure since a logical error satisfies the client's failure criteria.
The model quits.	⭐	This is a phenomenal stump!
The model is unable to complete computationally intensive calculations (for example, failing at RSA).	❌	We want to achieve failures in mathematical reasoning, not just failures in computation.

How to Write Great Proofs

Goals:

Submit high-quality, original proofs that can be used to teach leading AI models the best practices for mathematical proof writing.

- See below for a rubric on what we are looking for in proofs, and see this guide (also on the Slack page) for advice on writing proofs.
- **DO NOT** use an LLM to generate or improve your proof, or copy and paste from an external source. You must be the sole author of your proof.

Lean towards providing more justification, while still being as concise as possible.

- A good rule of thumb is: imagine you are trying to write this proof for a diligent student, whom you are teaching about your subdomain, and who is insistent on the details and intricacies of an argument.

Avoid:

Common errors to be added here…

Checklist for Proof Statements	
Requirement	**Detail**
Proof must be correct.	Hopefully this is obvious! 🙂 In this project, we have very high standards for the proofs we are submitting. Reviewers will have to reject your task if there are any mathematical errors in your proof, even minor ones.
Proof must be clear and carefully justified.	Your proof should be clearly written, with no ambiguity. If reviewers cannot fully understand your proof, your task will be rejected. Your proof should also be carefully justified. It is fine to reference sources, use existing results, and use commonly accepted statements with little justification, but be specific about what result you are using and (if applicable), where it is found in your source.
Proof must be well-written.	Your prompt should be written with correct spelling, proper grammar, and proper KaTeX formatting. See this guide (on the Phoenix Canvas page) for advice on writing in KaTeX. After you submit your proof, make sure you reread it to check for typos and formatting issues.
Proof must be original.	Your proof should not appear verbatim in any research paper, textbook, or online forum (e.g., Math Stack Exchange). You must be the sole author.
Proof must not be written by LLM nor written with the assistance of LLM.	We cannot accept any use of LLMs in our project. Again, you must be the sole author of your proof.

Responding to Reviews

In this project, reviewers are not just checking your tasks to make sure the final answer is correct – their job is much more complicated than that! Because of this, the review process on this project may feel different to the review process you have gone through on other projects.

- Just like on other projects, reviewers need to ensure that your task is high quality and that it meets the other requirements we have from the client regarding difficulty, originality, and domain.

- Since our main goal is to teach AI models how to write proofs, reviewers are not just checking that your proof and "final answer" are correct – they are also checking that your proof is high quality.

- If a reviewer determines that your task is ready to be approved, they will submit your prompt and proof statement on the client's annotation platform and generate new model responses. Then, they need to determine if the new models are correct or incorrect and write several paragraphs evaluating and comparing the new responses.

- When reviewers are submitting your prompt and proof statement on the client's annotation platform, they are copying and pasting your work, and are not supposed to make any changes. To help them approve more tasks, reviewers can correct minor errors like typos or formatting issues, but they are not allowed to fix any mathematical errors or make any changes that alter the meaning of your prompt or proof statement.

Below are some general goals to keep in mind when responding to reviews.

Goals:

Reduce the number of review cycles.

- Unless you truly think a reviewer's comment is incorrect or based on a misunderstanding, lean toward incorporating their feedback.
- After you incorporate feedback, reread your task quickly to check for any typos or errors that you and the reviewer may not have caught the first time.

Be nice!

- We are all on the same team! Reviewers are trying to help you improve your task so it meets the client's standards. They are also trying to teach you how to write high-quality tasks that meet the project requirements, so that your tasks require fewer review cycles in the future.

- Reviewers are chosen carefully for their mathematical abilities and tasking skills. Their opinions are valid.

Avoid:

If a reviewer asks you a question, do not just answer the question in the comments – be sure to make changes to your task as well.

- Reviewers are not only trying to understand your task, they are also trying to help you improve your prompt and proof to meet the project's high standards for quality.
- Even if you think a reviewer has misunderstood something about your prompt or proof and their question does not directly apply, that often still indicates that something about your prompt or proof is unclear and can be improved.

Please do not argue with reviewers about whether something requires more justification or clarification.

- Just as you have been selected for this project because you have proven your strength as a graduate-level mathematician, reviewers have also been carefully selected for their mathematical strength and their skill at tasking. If a skilled mathematician reads your proof and is confused about something, this is almost always a sign that your proof needs to be clarified or justified further.
- Reviewers are not just checking that your proof is correct– they also need to understand your prompt and proof well enough to be able to evaluate new model responses that are generated when they submit your task on the client's platform.

Note:

Reviewers should also be nice and treat you with respect. If you feel that a reviewer is being unfair, not following the project guidelines, or there is some other fundamental issue you can't resolve, please submit a dispute form.

- **[Dispute form link]**

⏩ 1.3 Tips and Tricks

This is a living document. Last Updated 11/11/25

How to Produce High Quality Work, Fast!

Here are some tricks you can use to produce fantastic tasks fast and make it easier for reviewers to approve your submissions. **These are not project requirements – they are just suggestions that you can use if you want to.**

In general, certain tasks are particularly valuable because they require LLMs to make decisions. This introduces an additional *potential* point of failure for the model, making it easier to analyze its responses. Also, anything you can do to make your task easier to read and understand will help it get reviewed more quickly.

1. Use the Prompt Format: "Prove or Disprove…"

These tasks require the model to determine whether a given statement is true or false and provide a corresponding proof or counterexample. They typically take the form: **"Prove or disprove: [mathematical statement]."** Some examples:

- Prove or disprove: there are finitely many prime numbers.
- Prove or disprove: the rings $k[t]_{(t)}$ and $k[[t]]$ are isomorphic.
- Prove or disprove: if $\{f_n\}$ is a sequence of square integrable functions satisfying $\lim_{n\to\infty} f_n = f$, then $\lim_{n\to\infty} \int_0^1 f_n dx = \int_0^1 f dx$.
- Let M be a smooth manifold, and ∇ a connection on TM. Prove or disprove: there exists a Riemannian metric g such that ∇ is Levi-Civita connection corresponding to g.
- Let $f : X \to Y$ be a morphism of schemes. Prove or disprove: if f is proper with finite fibers then f is finite.

2. Use the Prompt Format: Existence of an Object

These tasks require the model to determine whether a certain mathematical object can exist. They typically take the form: **"Prove whether or not there exists a mathematical object X satisfying [condition a_1], … , [condition a_n]"**, where each a_i specifies a property of or constraint on X. Some examples:

- Prove whether or not there exists a function integrable on $[0, 1]$, but not square integrable on $[0, 1]$.
- Prove whether or not there exists a scheme which is both affine and proper over $\operatorname{Spec} k$.
- Prove whether or not there exists a commutative ring which is both Noetherian and has infinite Krull dimension.
- Prove whether or not there exists a topological space which is non discrete, Hausdorff, and finite as a set.
- Prove whether or not there exists a smooth manifold without boundary which has every Stiefel-Whitney class $s_i(TM)$ equal to zero, but does not have trivial tangent bundle.

3. Write Concise Tasks, but Understand Your Audience

For reviewers' sake, be precise and concise when constructing your prompts and solutions. If you are able to make a shorter task and stump the model, reviewers will have an easier time reading the task and approving it!

That being said, do not omit important details just to make your proof shorter. Write your task with the understanding that your reviewer may not be intimately familiar with your preferred mathematics subfield. Recall that as a rule of thumb, ***all tasks should be accessible enough so that a generalist math PhD can understand and review them within an hour.***

We recognize that this is a delicate balancing act!

🏆 1.4 Golden Task Examples

This is a living document. Last Updated 11/11/25
[in progress!]

Video Examples of Golden Tasks

Click any link below to watch video walkthroughs of great tasks!

Analysis
- **Real Analysis**
- Complex Analysis
- Functional Analysis
- Measure Theory
- Ordinary Differential Equations (ODEs)
- Partial Differential Equations (PDEs)

Algebra
- Group Theory
- **Commutative Algebra**
- **Noncommutative Algebra**
- **Homological Algebra**

Number Theory
- Algebraic Number Theory
- Analytic Number Theory
- Combinatronics

Geometry
- **Algebraic Geometry**
- Differential Geometry

Topology
- **General (Point-Set) Topology**
- **Algebraic Topology**
- Differential Topology
- Low-Dimensional Topology

Logic & Foundations

- Set Theory
- Mathematical Logic
- Category Theory

Probability & Statistics

Other

🧪 1.5 Research-Level Tasks

Review-Friendly Research-Level Task Guidelines

First, let us clarify what we mean by *"research-level."* By this, we refer to a task whose primary focus lies outside the scope of standard graduate coursework: topics that are unconventional, highly specialized, or insufficiently covered in mainstream graduate-level textbooks. In other words, research-level work concerns areas where the literature is limited and the subject matter is niche or emerging.

The Phoenix math team values these tasks and sees them as a meaningful contribution to the project. At the same time, the team understands that they can take considerable effort to review. If they are not written clearly, fellows may wait a long time before receiving detailed feedback. To help ensure a smooth process, let's look at how to prepare these tasks properly.

Example: Lawson Homology Task

Lawson homology, introduced by H. B. Lawson and further developed with E. M. Friedlander, is a specialized homology theory for algebraic varieties that connects algebraic cycles with topological methods. It studies families of algebraic cycles parametrized by spheres, producing homological invariants that capture both geometric and topological information about a variety. Although it is a relatively niche topic within algebraic geometry, known and studied by a comparatively small community, it provides an important bridge between classical cycle theories (such as Chow groups) and topological homotopy techniques, offering deep insights at the interface of algebraic geometry and homotopy theory.

Let $X = \{(x, y, z) \in \mathbb{A}^3_{\mathbb{C}} : xy = z^2 - 1\}$. Prove or disprove: The path connected component of $\mathcal{Z}_1(X)$ containing the zero-cycle, equipped with the Chow topology, has the homotopy type of $\mathbb{CP}^\infty$.

This task looks at the space of all algebraic one-dimensional subvarieties (that is, algebraic curves) on a certain affine quadric surface. This space, called the one-cycle space, includes formal integer combinations of such curves and is given the natural Chow topology. The goal is to determine whether the connected component containing the zero cycle has the same homotopy type as infinite-dimensional complex projective space.

Now, let's walk through the solution. This is the heart of the task: a clear, detailed, and well-organized solution, even if lengthy, can be reviewed far more efficiently than a short answer that lacks details and references.

Let $L \subset X = \{(x, y, z) \in \mathbb{A}^3_{\mathbb{C}} : xy = z^2 - 1\}$ be the affine subvariety of X defined by $x = 0$ and $z + 1 = 0$. We know that $L \cong \mathbb{A}^1$ and we will show that $X \setminus L$ is isomoprhic to $\mathbb{A}^2_{\mathbb{C}}$. To see this, let's observe that $X \setminus L = D(x) \cup D(z + 1)$, so we can define the morphism $f\colon X \setminus L \to \mathbb{A}^2_{\mathbb{C}}$ by taking $(x, y, z) \mapsto (s, t)$, where $s = x$ and

$$t = \begin{cases} (z-1)/x, & (x, y, z) \in D(x) \\ y/(z+1), & (x, y, z) \in D(z+1) \end{cases}.$$

Observe that $(z - 1)/x = y/(z + 1)$ on $D(x) \cap D(z + 1)$ since we are working on X, so f is well-defined. The morphism f is invertible, with inverse $g\colon (s, t) \mapsto (s, st^2 + 2t, st + 1)$.

We begin by choosing a specific curve inside our surface, obtained by setting the first coordinate to zero and the third coordinate to minus one; this curve is isomorphic to the affine line. Our goal

is to show that, after removing this curve, the remaining surface is actually isomorphic to the affine plane.

To do this, we cover the complement by two standard open sets and define a map to the affine plane by taking one coordinate directly and defining the second coordinate differently on each open piece. We verify that these formulas agree on the overlap, so the map is well defined on the whole complement. We then write down a candidate inverse map, which suggests that the complement is isomorphic to the affine plane. The only thing left at this stage is to check explicitly that these two maps are indeed inverses of one another.

Let's get started by showing that g lands into $X \setminus L$. Clearly g lands into X as

$$\begin{aligned} xy &= s(st^2 + 2t) \\ &= s^2t^2 + 2st \\ &= (st+1)^2 - 1 \\ &= z^2 - 1. \end{aligned}$$

If, by contradiction, there exists $(s,t) \in \mathbb{A}^2_{\mathbb{C}}$ such that $g(s,t) \in L$, then we have

$$\begin{cases} s & = 0 \\ st + 2 & = 0 \end{cases},$$

which is absurd. An easy calculation now shows that f and g invert each other, so f is an isomorphism of varieties:

$$f \colon X \setminus L \cong \mathbb{A}^2_{\mathbb{C}}.$$

This step is important to document clearly: while the person writing the task may be able to verify the inverse in just a couple of minutes, the reviewer cannot assume it is correct without checking. By explicitly noting that the inverse remains to be verified, we make the argument transparent and save the reviewer from having to guess or reconstruct missing details.

Now, to study the homotopy type of $\mathcal{Z}_1(X)$ we are gonna use the Lawson Homology groups of X:

$$L_pH_k(X) = \pi_{k-2p}\mathcal{Z}_p(X).$$

Here $\mathcal{Z}_p(X)$ stands for the group of p-dimensional cycles of X, equipped with the Chow Topology (see Theorem 1.3.3 of **[2]**).

At this stage, we move from the geometric description of the surface to the topological study of its cycle space. To analyze the homotopy type of the space of one-cycles, we introduce the Lawson homology groups, which relate homotopy groups of cycle spaces to algebraic cycles. We also establish the notation we will use and cite the relevant references, so the reader and reviewer can quickly locate the precise definitions and results. This makes the argument transparent, allowing the reviewer to verify each step without having to reconstruct background material.

Now, as shown by Paulo Lima-Filho in Proposition 4.8 of **[1]** we have a long exact sequence in Lawson Homology

$$\cdots \to L_pH_k(\mathbb{A}^1_{\mathbb{C}}) \to L_pH_k(X) \to L_pH_k(\mathbb{A}^2_{\mathbb{C}}) \to L_pH_{k-1}(\mathbb{A}^1_{\mathbb{C}}) \to \cdots$$

Let's take $p = 1$ (we are studying 1-dimensional cycles!) in the sequence above and observe that , thanks to Example 1 of **[1]** , $L_1H_k(\mathbb{A}^n_{\mathbb{C}}) = \mathbb{Z}$ for $k = 2n$ and 0 elsewhere. We obtain

$$L_1H_k(X) \cong \begin{cases} \mathbb{Z}, & k = 2,4 \\ 0, & k \neq 2,4 \end{cases}.$$

In this step, we bring in a key tool from the literature: a long exact sequence in Lawson homology due to Paulo Lima-Filho. We apply this sequence to our situation and then use a known computation for affine space, also properly cited, to determine exactly which Lawson homology groups of our surface are nonzero. Since we are focusing on one-dimensional cycles, we specialize to the appropriate case. By citing these results clearly, we make it easy for the

reviewer to check the external facts we rely on and to follow how the homology groups of our variety are computed.

In other words, the homotopy group $\pi_n \mathcal{Z}_1(X)$ is isomorphic to $\mathbb{Z}$ for $n = 0, 2$ and equal to 0 elsewhere. As the π_0-homotopy group of $\mathcal{Z}_1(X)$ is isomorphic to $\mathbb{Z}$, our cycle space has infinitely many path connected components, and each of them is homeomorphic to the connected component containing the zero-cycle. Indeed, as shown in in Theorem 1.3.3 of **[2]** , the cycle space $\mathcal{Z}_1(X)$, equipped with the standard addition of cycles, is a topological group homotopically equivalent to a CW-complex.

The path-connected component of $\mathcal{Z}_1(X)$ containing the zero-cycle, denoted by $\mathcal{Z}_1(X)_0$, is therefore a path-connected CW-complex with nonzero homotopy group in dimension 2 only, hence thanks to Theorem 3.7 of **[3]** and the Whitehead Theorem, we get

$$\mathcal{Z}_1(X)_0 \simeq K(\mathbb{Z}, 2) \simeq \mathbb{CP}^\infty.$$

Providing clear references is essential in a task like this. It shows exactly which results are being used, lets the reviewer verify each tool quickly, and removes any ambiguity about assumptions or definitions. Even if the author knows these facts well, the reviewer cannot be expected to guess them or reconstruct them from memory. By citing precise sources and writing transparently, you make your beautiful “research-level” task much faster to review — which means you get strong, timely feedback instead of delays.

[1] https://eudml.org/doc/90175

[2] https://www.researchgate.net/publication/392733540_An_Introduction_to_Lawson_Homology_and_Algebraic_Cycle_Spaces

[3] https://www.few.vu.nl/~trt800/theses/abeljuhasz.pdf

Note: As we can see below, the Model initially hesitates over the meaning of the notation but eventually interprets it correctly from context, so everything proceeds smoothly. However, this moment of confusion highlights an important point: if the notation had *not* been clear to them, we would need to introduce and define it explicitly in the statement. Even when the author understands the notation, it's good practice to make it unambiguous so the model and the reviewers can follow without hesitation.

All Response 1 Response 2 **Response 3**

Looking at this problem, I need to first clarify the notation, as $\mathcal{Z}_1(X)$ could have several interpretations in the context of cycle spaces.

The most likely interpretation is that $\mathcal{Z}_1(X)$ denotes the space of effective 0-cycles of degree 1 on X (i.e., $\mathrm{Sym}^1(X) = X$). However, this would not contain the "zero-cycle" (empty cycle).

Given the context mentioning "the path connected component containing the zero-cycle," I believe $\mathcal{Z}_1(X)$ refers to the full space of 1-cycles (formal sums of curves) on X, where the zero-cycle is the empty formal sum.

? 1.6 Fellow FAQs

This is a living document. Last Updated 10/21/25

Project Phoenix: Frequently Asked Questions

🚀 Onboarding & Platform Access

Q: How do I begin tasking? I keep seeing "Domain Model Training."

- A: Watch this **video**!

Q: How do I know if my practice task has feedback or is approved?

- A: You can check the status in your "My Tasks" queue.
    - **Returned for Edits:** The task stage will show as "not started," and you can start the timer. There should be reviewer comments for you to address.
    - **Approved:** You will receive an email that your task has been approved and you have been invited to the Project Phoenix production project. The task stage will say "ready to deliver" or "delivered."

Q: Who do I contact if I lose access to the annotations platform, Canvas, Pronto, Deel, etc.??

- A: Reach out to Handshake AI Fellow Experience using this link:
    - https://support.joinhandshake.com/hc/en-us/categories/202711128-Student-Alumni/
    - Click 'Contact Support' in the top right hand corner of the webpage. Then, complete the chatbot flow to submit your request.

💳 Contracts & Payments

Q: How and when do I receive my contract?

- A: You will receive your contract *after* your practice task is approved.

1. Once approved, go to the Handshake AI Platform.
2. Click **"Continue"** on Project Phoenix (the production project, not the "Practice" one).
3. Click **"Issue Contract."**
4. Navigate to Deel and sign your contract.
5. Return to the Handshake platform to begin claiming tasks.

Q: Why is my contract failing to issue?

- A: This is likely because your location is not set to the US. If you are not a US citizen, you must have OPT or CPT to work on this project.
- Please refer to Work Authorization Eligibility Help Center article for more information.

Q: Who do I contact if I need to request a CPT/OPT Offer Letter?

A: Reach out to Handshake AI Fellow Experience using this link:

- https://support.joinhandshake.com/hc/en-us/categories/202711128-Student-Alumni/
- Click 'Contact Support' in the top right hand corner of the webpage. Then, complete the chatbot flow to submit your request.
- Please refer to Work Authorization Eligibility Help Center article for more information.

Q: How and when do I receive payments?

- A: We send payments every **Wednesday** for the previous week's work (Monday-Sunday). Bonuses are also paid on Wednesdays. Sometimes, hourly pay and bonuses may arrive as separate payments, but both will arrive on Wednesday.

Q: Will I be paid for my practice task?

- A: Yes, but this is issued as an Onboarding Bonus of $150, after you have successfully signed your Deel contract.

Q: Who do I contact for pay discrepancies/missing bonus?

- A: If you have a question regarding pay disputes please fill out the Payment Disputes Form. This will initiate a ticket with our pay team, who will reach out to you via email.
- Note: Pay disputes are worked on in chronological order. Thank you for your patience as we work through these.

✍️ Tasking Guidelines

Q: What does "Graduate Level" mean for tasks?

- A: This guideline is primarily about finding problems that **stump the AI models**. The main goal is to fail the models. If the models are failing, the labs are interested in the

data. We will not reject tasks *only* for "not being graduate level." Your focus should be on challenging the models.

Q: What's a good strategy to get tasks approved quickly?

- A: Provide **concise solutions**. Aim to keep your solution to a single paragraph if possible, as this makes it easier for reviewers to assess and approve your work. For more details, refer to the "Tasking Strategies" and "Golden Task Examples" sections in the instructions.

Q: Can I include multiple proofs in one task?

- A: No. Each task must include **only one** proof statement. Prompts with multiple proof requests will be returned for edits.

Q: My task uses an image. What's a common reason for rejection?

- A: Tasks will be returned if the model's failure is due to *misinterpreting the image* (a perception error). The image must be clear, and the model must understand its content before **failing on the reasoning or logic** of the task.

Q: Does rephrasing a common textbook problem count as an original task?

- A: No. Minor wording changes to well-known problems (e.g., classical theorems) do not count as original.

Q: Where can I find definitions of a "valid model stump"?

- A: The "Instructions Document" on Canvas has a dedicated tab titled **1.2 Tasking,** which provides clear definitions and examples.

📊 Performance & Quality Standards

Q: Is there a limit on how many tasks I can claim?

- A: Yes. Fellows can claim up to **10 tasks per 48 hours** until they achieve five approvals on the first try.

Q: How is the task limit removed?

- A: The 5-task-per-day limit is automatically removed once you achieve a 50% approval rate.

Q: Are task/review incentives (bonuses) still active?

- A: Total time on the project is still active, and "Star Fellow" incentives remain active. To become a Star Fellow, you must achieve an 80% approval rate. See the **Leaderboards** for more info.

Q: What are the performance standards for Fellows?

- A: Performance is reviewed weekly based on the approval rate of your 5 most recent tasks.
    - **Offboarding Criteria:** An approval rate **below 30%** can result in offboarding.
    - **Promotion Criteria:** An approval rate **above 80%** makes you eligible for promotion to a Reviewer role.

Q: What are the standards for Reviewers?

- A: Reviewers who complete **fewer than 20 reviews per week** may be transitioned back to a Fellow role (with flexibility for schedules). Pod leaders also have discretion based on quality standards.

Q: What happens if I keep resubmitting tasks without addressing feedback?

- A: Resubmitting tasks without making the requested changes, or with only superficial edits, can result in being **offboarded from the program**.

Q: What should I do if I disagree with a reviewer's feedback?

- A: You are encouraged to disagree with feedback you believe is incorrect, but use this option thoughtfully. The best way to resolve this is to **attend office hours** and discuss the task with the team.
- The Reviewer Dispute form is available in the Canvas modules for fellows who have passed practice.

🏆 Leaderboard & Achievements

Q: Why did my total hours on the leaderboard decrease?

- A: A time-tracking audit was completed in late October to ensure the leaderboard shows **total paid hours (timer time)**. Discrepancies were corrected, which may have resulted in a decrease for some users as the data was corrected to reflect actual paid time.

Q: I was paid for an achievement, but after the audit, my new total means I no longer meet it. Do I have to pay it back?

- A: No. You will not have to pay back any money. You have already been paid for that work.

Q: Will I get paid again if I re-earn an achievement I was already paid for?

- A: No. The team will ensure each achievement is only paid once. If you earn an achievement in the future that you have already been paid for, you should not expect another payment for it.

Q: How often does the leaderboard update?

- A: The leaderboard updates every hour.

💬 Sessions & Support

Q: Where do I find the Zoom links for all the sessions (Office Hours, Webinars, etc.)?

- A: All links for these sessions are located in the **Canvas Calendar**.

Q: How does the Math Task-a-thon work?

- A:
  - **Bring:** A brand-new (not submitted) task statement and solution pair that "stumps" at least one model.
  - **Pitch:** You will have 3-5 minutes to present your task.
  - **Feedback:** You will receive live, actionable feedback from the team.
  - **Submit:** After the session, you polish your task and submit it.

Q: I missed a workshop/webinar. Can I watch a recording?

- A: Yes, past sessions are recorded and available in the **Canvas modules**. Task presentations are also shown in the "**Golden Task Examples**" section of the instructions.

For general support regarding Handshake AI

Please reach out to Handshake AI Fellow Experience using this link:

- https://support.joinhandshake.com/hc/en-us/categories/202711128-Student-Alumni/
- Click 'Contact Support' in the top right hand corner of the webpage. Then, complete the chatbot flow to submit your request.

➕ Referrals

Q: Who do I contact for issues with referrals?

- A: Reach out to Handshake AI Fellow Experience using this link: https://support.joinhandshake.com/hc/en-us/categories/202711128-Student-Alumni/

- Click ‘Contact Support’ in the top right hand corner of the webpage. Then, complete the chatbot flow to submit your request.
- Reference: Referral Help Center article.

2.0 Reviewer Playbook

This is a living document. Last Updated 10/25/25

Main Menu

Click any title to go to the corresponding tab

Document Tab	Contents
2.1 How to evaluate fellow tasks in Platform 1 🟩	• Technical Instructions • Task Evaluation Instructions • Task Rubrics
2.2 How to evaluate model responses in Platform 2 🟫	• Technical Instructions • Task Evaluation Instructions • P2 Model Rubric
2.3 Reviewer Pod Resources 🔮	• Understand Review Pod Structure • Schedule 1:1s • Pod Tracker (Your reviews/Pod Activity)
2.4 Golden Review Video Examples 🏆	• [in progress…]
2.5 Reviewer FAQs ❓	• [in progress…]

Welcome to the Project Phoenix Reviewing Team! If you've been contacted directly via email, you've been selected based on your consistently high-quality work. If you're new and haven't received this communication yet, you're not a reviewer—unless you're joining as a specialist.

Goals of the project

Submit high quality original proof statements and solutions to the AI Lab, and grade two model responses on their attempts to solve the problem. Grade attempts based on the correct reference answer to the prompt. Provide quality feedback to the AI Lab on the correctness, strengths, and weaknesses of their models.

Reviewers: Your Role

- Evaluate the quality of submitted prompts and tasks.
- Provide detailed evaluations and feedback to fellows.
- Submit Approved Submissions to the AI Lab's Platform and offer high-quality feedback.
- Meet weekly with your pod leader to align and receive support.
- Be active on Slack; each reviewer belongs to a pod, led by a lead reviewer, with its own Slack channel for communication and resources.

Benefits

- Immediate feedback and support on your reviews and tasks
- Direct coaching from STEM experts
- Future opportunities to grow with the Handshake AI team

Onboarding + New Reviewer Actions

New to the project and reviewing (new specialists or reviewers transferred from other projects)

1. Complete task as fellow task is approved
2. Complete first review but do not submit. Send to pod Slack channel for review feedback.
3. Feedback on review with pod leader
4. First review approved by other reviewer! You have access to unlimited reviews

Was a fellow, now promoted to a reviewer

1. Complete first review but do not submit. Send to pod Slack channel for review feedback.
2. Feedback on review with pod leader
3. First review approved by other reviewer! You have access to unlimited reviews

Expectations (once onboarding + Actions above are complete)

- Complete at least 20 reviews per week (both rejections and approvals count toward total reviews, ~15 hours total)
- Maintain consistent quality and timeliness in reviews
- You may only have 3 tasks claimed at a time and one timer running at a time.
- Communicate proactively with your pod leader if you can't meet weekly goals

Failure to meet expectations may result in a transition back to the fellow role, to ensure project quality and progress. **Please see section 2.3 "Pod Resources"** for more information about your role, benefits, and expectations!

🟩 2.1 How to Evaluate P1 tasks

This is a living document. Last Updated 11/5/25

This Tab is meant to guide you through all you need to know about reviewing on P1. We will first discuss the purposes of P1 at a high level. Second, we will explain the systems and processes you need to use P1 successfully and provide a step-by-step guide to reviewing tasks. Third, we will explain *how* you should review tasks in this project and explain the quality criteria and guidelines. In this section, we provide additional explanations and rubrics to help you answer the many questions you may have while reviewing tasks.

If we have left anything out, please reach out to the team!

Menu

Click any title to go to the corresponding section

Section	*Topics*
Overview & Goals of P1	• Goals • General Guidelines

Technical Instructions

Step-by-step Process and Walkthrough	• P1 • Claim sheet

P1 Task Evaluation Instructions

General Reviewing Guidelines	• Teach and Encourage Fellows • Reduce Revision Cycles • Approve Tasks and Don't Nitpick

How to Evaluate Prompts/Proof Statements [Rubric - click me]	• Originality • Difficulty/Level • Trick questions • KaTeX • Language & Clarity
How to Evaluate Model Responses & Failures [Rubric - click me]	• Proofs Produced • Valid Model Stumps
How to Evaluate Proof Solutions Given by Fellows [Rubric - click me]	• Level of Detail & Specificity • LLM Use • References • KaTeX • Language & Clarity

Overview and Primary Goals

As a reviewer in Platform 1 (P1), your primary goal is to evaluate tasks submitted by fellows, ensure they meet the project requirements, and approve tasks. You should never forget that the goal is to approve tasks and increase the number of approvable tasks over time (i.e., help teach fellows where they may have shortcomings so they can correct mistakes and improve). You should only approve tasks that you believe meet the project's quality requirements.

Getting more specific, your primary goals in P1 are to review the following 3 main fields before approval:

1. Prompts, a.k.a. proof statements given by fellows
2. Model failure(s)
3. Proof Solutions given by fellows

While we *do* care about the other task fields, you should rarely return tasks for issues with other fields alone. These three are the most important because:

1. Prompts: We send these directly to the lab.
2. Model Failure(s): At least one model failure in P1 indicates this problem meets the difficulty threshold for the lab. However, we do not send these model responses to the lab.
3. Proof Solutions given by fellows: We send these directly to the lab.

Technical Instructions

Step by Step Process (systems + how to use the platforms)

Helpful References

- **[Video Guide]**
- **[Claim Sheet]**

How to claim tasks

- You should always claim tasks using the claim sheet **[Claim Sheet]**.
- Only work on 1-2 reviews at a time, and complete your reviews within 24 hours of claiming a task. Claims will be removed after 24 hours if no action has been taken.
- If you have claimed a task by accident or no longer wish to review a specific task, you can release the claim on a task by clicking the "Exit" button in the top right, and then choosing "Abandon Task."

How to add feedback in P1

- When reviewing a task, you have the option to add four different types of comments: Major Error, Minor Error, General Feedback, and Praise.
- If you think a task needs to be returned, at least one of your comments must be a "Major Error" or "Minor Error." If you think a task is ready to be approved, then you cannot have any "Major Error" or "Minor Error" comments.
- If you think the fellow is submitting high-quality work and could be a good reviewer, add a "praise" comment and compliment them.

How to approve a task

- Before approving a task on P1, you must submit the task on P2 and evaluate the model responses. **[link to "Part 2" section]**

How to report LLM usage or other bad behavior

- If you suspect that a fellow is using LLM in their solutions, or you have serious concerns about their ability to deliver tasks that are in line with the project needs, please post in your Slack reviewer pod with:
    - The Task ID
    - The Fellow's name or email address
    - The reason
- See the How to Evaluate Fellows Proof Solutions section for tips on how to recognize LLM use. **[Link to LLM guide]**

- Send the task back to the fellow with comments on the issues you see, saying you suspect LLM use. Do not tell the fellow that you are reporting them.

Task Evaluation Instructions

General Reviewing Guidelines

Goals:

Teach and Encourage Fellows

- Be encouraging and kind! Teach the fellows so that their work improves over time.
- Remember your best professors? Try to be a bit like them :)

Reduce Revision Cycles

- Review tasks top to bottom before making decisions. We want to ensure tasks do not go through entire review cycles for each minor issue. Please return tasks only after you have raised all issues in the task's comments.
- Please leave general feedback for the next reviewers if there was a gap in the solution that took a while to verify, so that the next reviewer does not need to take time to repeat the verification.

Approve Tasks and Don't Nitpick

- If a task meets all acceptance criteria but you see room for improvement, please do not send the task back.
- Even if a task has minor errors (i.e., typos or formatting issues that do not change the mathematical meaning), you should still approve the task and fix these errors when submitting the task on P2.
- Since our other goal is to teach fellows, if you are approving a task but it has minor errors or you have suggestions for improvement, try to leave a "general feedback" comment that lets the fellow know what you fixed or what you'd suggest doing next time.

Avoid:

Common errors to be added here…

How to Evaluate Prompts/Proof Statements

Goals:

Ensure Problem Originality

- Problems must be original (not copied and not simply reworded).
- Problems can be "inspired" by well-known results, but should have different constraints or assumptions.
- If there is a research paper or proof that specifically inspired the task, you should expect a reference to this, whether in the solution or in the reference section of the task.

Ensure Correct Difficulty Level

- We accept any undergrad- to research-level material as long as it stumps the model(s).
- Tasks **must** require math expertise (knowledge beyond high-school math) and should not be IMO or contest-style problems (especially for practice tasks).
- Problems must not be trick questions.

Ensure Clarity

- Prompts must be clear and understandable. However, if the statement is strong but you see room for improvement, please do not send the task back. We strive for perfection, but should not let this come at the expense of approval rates.
- Clarity means using correct formatting: fellows must use proper KaTeX, but you can (and should) fix minor errors that don't change the meaning of the statement when you take the task to P2.

Avoid:

Do not send tasks back just because the prompt doesn't use "prove or disprove."

- Prompts need to ask for a proof explicitly, but they do not have to use "prove or disprove specifically." Fellows can use "prove that," "prove or disprove," "demonstrate/show with proof," or other similar statements in their prompts.

Rubric: How to Evaluate Fellow Prompts/Proof Statements		
Error	**Approve ✅ or Reject ❌**	**Reason**
Minor KaTeX errors	✅	If there are minor KaTeX issues that you could fix without changing the logic/reasoning/structure of the statement, please do so when you take the task to P2. We want to try to approve tasks as much as possible. (Ex: you should fix "Solve for x" → "Solve for x")

Minor grammatical issues or typos	✅	If there are minor grammatical issues or typos that you could fix without changing the logic/reasoning/structure of the statement, please do so when you take the task to P2. We want to try to approve tasks as much as possible. (Ex: you should fix “Find the derivate” → “Find the derivative.”)
Language & Clarity	❌	If the prompt is unclear to the point that you or the models do not fully understand the question, please reject. We want to ensure the models in P2 do not fail due to ambiguity.
Ambiguous definitions	❌	Similarly to above, we want the model to be stumped by a logical issue. We don’t want stumps that occur due to ambiguity in the prompt. Fellows must specify the definitions used in their prompt. Some common examples: - “Manifolds” may not include “manifolds with boundary”. - “n-differentiable functions” standing for C^n-functions.
Mathematical issues or typos	❌	If there are mathematical issues or typos that **do change** the logic/reasoning/structure of the statement, please send the task back. (Ex: send back a task if the prompt states “Let f be a real-valued function on $[0,1]$” when the statement actually requires f to be a function from $[0,1]$ to $[0,1]$.)
Wrong difficulty level	❌	If a prompt is an IMO or contest-style problem, or it can be solved with only general knowledge (meaning, it does not require any undergraduate college-level math to solve), then please send the task back. We want to focus on tasks that require mathematical expertise. Though we ask for PhD level, we actually can accept anything above US high school level if it stumps models.
Not original	❌	If a prompt is copied word-for-word from any source, or if it is a simple restatement of a well-known theorem or exercise, please send the task back. Tasks can be inspired by existing problems, but we still need them to be original.
Wrong domain	❌	If a prompt falls under another field like physics or finance, please send it back and ask the fellow to provide all necessary context and define all terminology in purely mathematical terms. Prompts must be solvable with only math knowledge.
Trick question	❌	If a prompt asks the model to prove something that is impossible (without giving it the option to disprove the statement), please send the task back. We want the models to fail because of genuine mathematical reasoning errors, not because they have been tricked.

What is a “minor error” that you can edit when submitting on P2?

In general, if the only issues with the task are small grammatical errors, formatting errors, or typos that do not change the meaning of what is written, then you can and should approve the task and fix these errors when submitting the task on P2.

- You will not be able to edit the prompt or proof statement in P1 – that is fine! Just leave the fellow a "General Feedback" comment that explains, in a nice way, what errors you fixed so that they can know for next time.
- Below are some examples to help clarify what we mean by a "minor error."

When to fix a task yourself

- Grammar and typos:
  - You can take care of spelling or punctuation mistakes.
  - Example: "Find the derivate" → "Find the derivative."
  - Example: "Let A be an matrix" → "Let A be a matrix"
- KaTeX and notation issues:
  - You can fix display or formatting errors (missing $, \ , or {}), and correct subscripts or indices.
  - Example: v_01 → v_0, v_1
  - Example: "Solve for x" → "Solve for x"
- Minor wording or clarity fixes:
  - You can add or adjust short words to make a sentence precise, as long as it doesn't change the math or the format too much.
  - Example: "any such region looks like X" → "any such region is contained in a region that looks like X"

When to send tasks back for fellows to edit

- Do not make any change that alters what the prompt/solution says or assumes. If there are errors that change the meaning of what is written, send the task back to the fellow.
- Changes you should **not** make yourself:
  - Example: "A is a commutative ring" → "A is a commutative ring with unit" ❌
  - Example: "$\pi_1 = 0$" → "All homotopy groups are zero" ❌
  - Example: "Compute the determinant of A" → "Show that A is invertible" ❌

How to Evaluate Model Responses & Failures

Goals:

Ensure models produced mathematical proofs

- If models did not produce proofs and the prompt did not explicitly ask for a proof, ask a fellow to update their prompt (by adding "prove or disprove," "prove," "disprove," "show that," etc.).

Ensure model(s) produced valid stumps

- Prompts must stump the models on P1 to ensure that they are at the right difficulty level.
- For practice tasks, we need at least 2 stumps, and for production/poindexter tasks, we need at least 1 stump.

Avoid:

Rejecting tasks for nitpicking fellow errors

- If the model failure requirements were met, do not send tasks back if fellows have just mislabeled which models failed, or misdescribed the model failures. The only purpose of these fields ("Which model response failed to solve the proof?" and "Explain in 2-3 sentences why the model response was wrong.") is to help reviewers ensure that model failure requirements were met.

Rubric: How to Evaluate Model Responses & Failures			
Model Error	**Valid Stump (Approve ✅) or Invalid Stump (Reject ❌)**	**Reason**	**Templated response to fellow (if sending back)**
Wrong citations, incorrect knowledge recall, or model reference is wrong	✅	This stump is acceptable. We do not want the models to reference incorrect or false sources, and we do not want the models to use false information to justify steps in a proof.	
Images being misread	❌	We are looking for reasoning errors that occur after correct interpretation of the problem. We want the failure to be logic-based and not due to multimodal processing.	This model failure occurs due to image processing. In order to be considered a valid stump, the model must fail AFTER correctly interpreting the image.
Small computational errors (as long as they don't cause the overall proof to be incorrect)	❌	Sometimes the models make small typos within the proof that they do not use in the remainder of the proof. So there is no true logical error in this case.	While there was a small computational error in one part of the proof, the model self corrected, and the

		Example: -In one line, the model incorrectly says $a-b$, but in the remainder of the proof, it uses the correct quantity $a+b$.	remainder of the task was correct. This does not count as a valid model failure.
Rambling: Model says something incorrect first, then corrects self	❌	The end result of the model reasoning is most important. As long as there is a valid proof at the end of the model's response, this is not a valid model stump.	The initial portion of the response is the model "thinking." However, the model presents a valid proof at the end, so this is not a valid model failure.
Rambling: Model says something correct first, then changes to something incorrect	✅	The end result of model reasoning is most important.	
The model gives a strong counterexample that could work, but the explanation is incorrect or incomplete.	✅	Because we are concerned with proper justification and not just the correct conclusion, a valid counterexample without appropriate justification is considered a valid stump.	
Ambiguous definitions - fellows specify the definition used in the statement/prompt, and the model uses an incorrect definition in its proof.	✅	If the prompt and associated definitions are clearly provided and the model still uses the incorrect definition, this is a valid model failure.	
Ambiguous definitions - fellows DO NOT specify the definition used in the statement/prompt, and the model uses an incorrect definition in proof.	❌	We want the model to be stumped by a logical issue. We don't want stumps that occur due to ambiguity in the prompt. Some common examples: - "Manifolds" may not include "manifolds with boundary". - "n-differentiable functions" standing for C^n-functions.	If you are using a nonstandard definition or a term whose definition varies across sources, please make sure that it is provided in the prompt. We do not want the model to fail because it uses a different definition than you.
The final answer is correct, but not enough detail in the proof - reviewer discretion.	Gray area - reviewer discretion	We want the model to achieve a good balance between sufficient detail without overexplanation. Egregious lack of detail is a problem, but ultimately, reviewers will decide this. In general, it is okay for the model to use commonly accepted statements with little justification. We understand that this can be a gray area. Use your best judgment as a reviewer.	While there are some steps which were not fully justified, the skipped sections are results that are well-known enough to not require further justification. The rest of the proof is acceptable, so this is not a valid stump.
Proof final answer is wrong	✅	This is an acceptable failure since the model gives an incorrect response.	

Proof final answer is correct, but uses incorrect logic	✅	This is an acceptable failure since a logical error satisfies the client's failure criteria.	
The model fails due to a lack of prompt clarity.	❌	We want to ensure that the model fails due to logic and reasoning error and not due to an ambiguous or unclear prompt.	This model failure occurs due to an issue with parsing the prompt. Please revise the prompt and ensure that the model still fails.
The model quits	⭐	This is a phenomenal stump!	
The model is unable to complete computationally intensive calculations (for example, failing at RSA)	❌	Again, here we want to achieve failures in math/proof logic, not just computation.	While the models may have been stumped, we have to stump them based on incorrect proof, justifications, or mathematical logic. Here, the models have failed due to a lack of computational power rather than faulty logic.

How to Evaluate Fellow Proof Solutions

Goals:

Ensure no LLM Use

- Do your best to recognize patterns that LLMs use, and make sure the proof is not an LLM-generated response. Below this section is a guide for how to tell if a fellow's proof is Questionably 🟡, Highly Likely 🟠, or Definitely 🔴 LLM (See the guide below this section's rubric).

Ensure Appropriate Level of Detail

- We want to achieve a good balance between proofs providing sufficient but not excessive detail. We recognize that this can be a grey area, and ask for your best judgment as reviewers.
- Fellows are not expected to explain everything - they only have two hours! Likewise, as reviewers, we do not want to read entire research papers in a single task.

Ensure Clarity

- Proofs must be clear and understandable. If the proof is correct but you see room for improvement (reordering steps, etc.), please do not send the task back—if a task meets the acceptance criteria, please approve it. We strive for perfection, but should not let this come at the expense of approval rates.
- Clarity means using correct formatting: fellows must use proper KaTeX, but you can (and should) fix minor errors that don't change the meaning of

the proof when you take the task to P2.

Avoid:

Common errors to be added here…

Rubric: How to Evaluate Fellow Proof Solutions		
Error	**Approve ✅ or Reject ❌**	**Reason**
Insufficient detail, but correct overall	Gray area	If the proof's final answer or overall reasoning is correct, but there is not enough detail in the proof, the decision is at the discretion of the reviewer. (Egregious lack of detail is a problem.) We want to achieve a good balance between sufficient detail and without overexplanation. In general, it is okay for a fellow's proof to use commonly accepted statements with little justification. (However, if an unknown result is referenced and an internet search does not help, more explanation is needed.) We understand that this can be a gray area. Use your best judgment as a reviewer.
Minor KaTeX errors	✅	If there are minor LaTeX issues that you could fix without changing the logic/reasoning/structure of the proof, please do so when you take the task to P2. We want to try to approve tasks as much as possible. (Ex: you should fix "Solve for x" → "Solve for x")
Minor grammatical issues or typos	✅	If there are minor grammatical issues or typos that you could fix without changing the logic/reasoning/structure of the proof, please do so when you take the task to P2. We want to try to approve tasks as much as possible. (Ex: you should fix "Find the derivate" → "Find the derivative.")
Language & Clarity	❌	If the prompt is unclear to the point that you or the models do not fully understand the question, please reject. We want to ensure the models in P2 do not fail due to ambiguity.
Mathematical issues or typos	❌	If there are mathematical issues or typos that **do change** the logic/reasoning/structure of the proof, please send the task back. (Ex: send back a task if the proof states "Let f be a real-valued function on $[0,1]$" when the statement actually requires f to be a function from $[0,1]$ to $[0,1]$.)
Reasoning errors	❌	If you feel that a fellow has made a reasoning error that renders their proof incorrect or incomplete, please send the task back.
Not original	❌	If a proof is copied word-for-word from any source, or if it is entirely a well-known theorem or exercise, please send the task back. We understand that all proofs will cite known results - this is okay. We want to avoid exact copies of proofs. Existing problems can inspire

		tasks, but we still need them to be original.
LLM Use: Highly Likely LLM 🟠 or Definitely LLM 🔴	❌	We cannot accept any use of LLMs in our project. If something is Questionably LLM 🟡, reach out to your pod and ask for help. Usually we can approve these edge cases, but always worth another perspective.

Fellow LLM Guide

Questionably LLM 🟡

- **a.** Overly formal, didactic tone – reads like a textbook summary instead of a working mathematician's scratchpad or argument.
- **b.** Pedagogical scaffolding – phrases like "Step 1: Recall the definition," "Step 2: Construct an example," "Step 3: Verify…" are very LLM-like, but some human expositors or teachers also use them.
- **c.** Verbose restatements of the problem – especially opening paragraphs that paraphrase the entire question before beginning the argument.
- **d.** Uniform paragraph rhythm – every paragraph of similar length, like a template being filled in rather than an organic flow.
- **e.** Explicit signposting – "In summary," "To conclude," "Therefore we have shown…" at the end of every section (common in LLM outputs, but not proof of it).
- **f.** Mathematically correct but stylistically bland – reads like a polished StackExchange answer rather than someone's original notes.

Highly Likely LLM 🟠

- **g.** The "key insight" sentence pattern – recurring phrasing like *"The key insight is that…"* across multiple responses is a strong tell. Sometimes, humans use this language too, but look out for specific repetitive phrases.
- **h.** Self-quotation or echoing the prompt – e.g., *"Thus, we have shown that the statement 'Every compact operator is normal' is FALSE."* This "third-person echo" is almost unique to LLMs.
- **i.** Over-explained trivialities – e.g., spending a paragraph redefining terms that any intended audience already knows.
- **j.** No quirks of notation, citation habits, or creative leaps. Everything sounds uniformly neutral and safe.

Definitely LLM 🔴

- **k.** Nonsensical rigor at the meta-level – perfectly grammatical but mathematically meaningless transitions ("Therefore, by linearity of the proof…").
- **l.** Contradictory or circular reasoning hidden under polish – the text "flows" beautifully but doesn't withstand close inspection.
- **m.** Including flawed reasoning and then correcting oneself in the proof, or including long descriptions of the brainstorming process ("Wait, that's not right. Let me try a different approach.").
- **n.** Mechanical paraphrasing of the question in quotes at the end – no working

mathematician quotes their own prompt verbatim.

o. Failure to use personal or community notation – human mathematicians often have idiosyncratic symbols, abbreviations, or references ("cf. Atiyah's argument"); LLMs default to generic forms.

p. Flawless grammar with zero personality – real experts make small style slips, switch between first-person plural and passive, or use humor; LLMs stay eerily smooth.

🟫 2.2 How to Evaluate P2 Model Responses

This is a living document. Last Updated 11/5/25

This Tab is meant to guide you through all you need to know about submitting approval-ready tasks in P2. We will first discuss the purposes of P2 at a high level. Second, we will explain the systems and processes you need to use P2 successfully and provide a step-by-step guide to uploading tasks. Third, we will explain *how* you should evaluate models in P2 as you submit problems to the Lab's platform. We will explain the quality criteria and guidelines for ranking model responses and providing feedback. In this section, we provide additional explanations and rubrics to help you answer the many questions you may have while submitting tasks to P2.

If we have left anything out, please reach out to the team!

Menu

Click any title to go to the corresponding section

Section	*Topics*
Overview & Goals of P2	• Goals • Quality • General Guidelines

Technical Instructions

Step-by-step Process and Walkthrough	• P2 • Claim sheet

P2 Model Evaluation Instructions

General P2 Guidelines	• Correctness assessment • Preference Ranking • Python • Issues generating responses

How to Fill out Correctness Assessment	
How to Fill out Preference Justification	
How to Rank Your Preferences	

Overview and Goals

As a reviewer in Platform 2 (P2), your primary goal is to evaluate the client's two additional model responses as they attempt to solve the problem we have approved as high quality in P1. The goal here is not to stump models; the goal is to evaluate the two model responses with specific reference to the techniques they use.

Specifically, you will:

- Copy and paste the P1 proof statement into P2, and generate two new model proofs
- Write high-quality paragraphs evaluating each response and comparing the two responses
- Use buttons to determine which of the two model proofs you like better
- Copy and paste the P1 proof solution into P2

Goals:

- Ensure that your written evaluations on P2 (the correctness assessments and the preference justification) are well-written and specific. **These answers are some of the most important data we deliver to the client, so it is crucial that they are high quality.**

Technical Instructions

Step by Step Process (systems + how to use the platform)

Helpful references:

- **[Platform 2 Video Guide]**
- **[Claim Sheet]**
- **[P2]**

1. **So you are about to approve a task - excellent!**

- Before we click the "approve" button in P1, we have to copy and paste the work into P2, then evaluate two more model responses.
- Keep your timer running during the P1 review session so we can make sure to pay you for the time you log while working in P2 (we can't track time in P2).

2. **Log in to P2**
    - By the time you are at this step, you should have access to P2
    - Enter your email address
    - Enter the code sent to your email address
    - Click "Agree to the terms and proceed to task"

3. **Copy and paste the proof statement/prompt from P1 into P2**
    - If there is KaTeX code used in P1, start by unformatting the text by clicking the **T_X button** on the top right corner of your screen. This allows you to cleanly copy and paste the text into P2.
    - If there is an image in the statement, please download and upload the images appropriately.
    - If necessary, make minor grammar, KaTeX, or wording changes.
        - See the guidance at the end of the "How to Evaluate Prompts/Proof Statements" section in Part 1 for what constitutes a minor error that you can fix.

4. **Click "Send" to generate two model responses**
    - It can take up to 5-8 minutes to produce the model responses
    - The models will finish, and choices will appear
        - **Do not fill out these choice buttons yet! Filling this out immediately will hide the proof you "prefer" less. We need to see both proofs for the next steps, though. Move to the next step:**
    - Occasionally, the models will time out and not produce a response, or will time out partway through producing a response. If this happens, refresh the webpage and submit the proof statement/prompt again.

5. **Enter the correct math content tags for the problem (the closest related subfield on P1)**
    - Scroll below the model responses in P2 and paste the P1 solution

6. **Copy and paste the reference proof (the fellow proof solution from P1) into P2**
    - If there is KaTeX code used in P1, start by unformatting the text by clicking the **T_X button** on the top right corner of your screen. This allows you to copy and paste the text into P2 cleanly.

- If necessary, make minor grammar, KaTeX, or wording changes.
    - See the guidance at the end of the "How to Evaluate Prompts/Proof Statements" section in Part 1 for what constitutes a minor error that you can fix.

7. **Evaluate the two model responses compared to the reference proof**
    - **See the section below ("P2 Model Evaluation Instructions") for more information.**
    - Provide 2-3 sentences on whether Model A is correct or incorrect, and which mathematical techniques it uses, and if they are the same as the reference answer. If there are issues, note these specific issues in the model responses here.
    - Provide 2-3 sentences on whether Model B is correct or incorrect, and which mathematical techniques it uses and if they are the same as the reference answer. If there are issues, note these specific issues in the model responses here.

8. **Enter your preferences and justification**
    - **See the section below ("P2 Model Evaluation Instructions") for more information.**
    - Write 2-3 sentences on why you chose one model response over the other.
    - Return to the model responses up top, and now rank your preferences for the two model responses.
        - The response you prefer less will disappear - this is ok!

9. **Click "Submit task and start new"**
    - Your P2 submission has been sent!!

10. **Return to Platform 1 or "P1" (Handshake AI Platform)**
    - Return to P1 and click "Approve."
    - Do not click "Claim Next Task" - this will assign you a random review.
    - Return to the claim sheet to claim your next task.

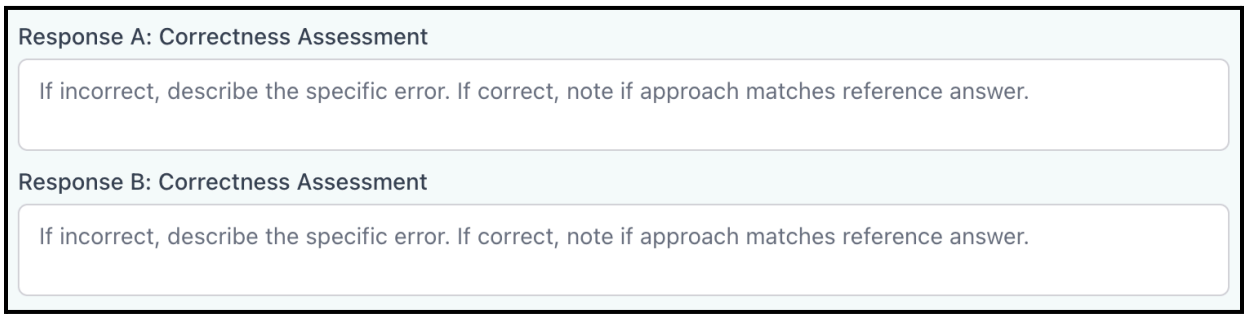
P2 Model Evaluation Instructions

General P2 Guidelines

- The written evaluations we submit through P2 (the correctness assessments and preference justification) are very important data for the client. Please take your time to provide well-written, detailed answers.

- It is fine if both models in P2 give correct answers! We tend to see both models in P2 give correct answers about 30-45% of the time. Our model requirement failures in P1 are designed to ensure that problems are at a sufficient difficulty level for the client.
- Models may produce python code as part of their responses. This is fine, as long as their response can still be considered a full proof.

How to fill out the "Correctness Assessment"



Please (1) clearly state whether the proof is correct or incorrect and whether the model's approach matches the reference answer, and (2) evaluate the proof's quality. More guidance on these two steps is below.

Step 1: Check for Correctness

Determine whether the proof (or response) is **correct** or **incorrect**:

- If it is **incorrect**, you should describe the **specific key error(s)** clearly (e.g., a wrong step, an unjustified claim, or a misuse of a tool). Please err on the side of descriptiveness here.
- If it is **correct**, state that and then evaluate the method used, and whether it matches the reference answer or not. Please err on the side of descriptiveness here.

Step 2: Evaluate Against Key Criteria

Assess its quality using the following six dimensions as guidance (**both if correct or incorrect**):

1. **Clarity**: Is it easy to follow, or does it feel confusing?
2. **Learning Value**: Does it teach practical techniques or tricks that help deepen understanding?
3. **Minimal Prerequisites/Tools**: Does it rely only on necessary prior knowledge and avoid excessive external machinery?
4. **Strict Rigor**: Does it justify every step and avoid unproven or hand-wavy claims?

5. **Conceptual Digestion**: Does it make the statement easier to internalize and really "click"?
6. **Approach**: Did it use the correct approach (compared to the reference answer)? If not, explain where it strayed and how.

Other Considerations

- It is fine if the models use Python, provided that their response is still a proof. For example: if the model proves that an identity is true as long as it holds for a specific finite set of integers, and then uses Python to check the identity for all integers in this set, this is fine. However, if the models use Python to check the cases $n=1$ to $n=1000$ for an identity, and then claim "it must be true for all natural numbers," this would be incorrect.

Examples of strong correctness assessments:

- *"The model attempts to construct counterexamples, but they are incorrect. On the first try, it attempts non-negative matrices but the inequality holds for non-negative matrices so it is incorrect. On the second try, it attempts matrices with both positive and negative entries, but the example does not work. The model then concludes by stating that the result is known for non-negative matrices, but not general matrices without any other proof."*
- *"Response A is correct, and is clearly written and well-organized. It uses a somewhat similar approach to the reference answer, but does not find the simpler way of writing R that the reference answer does. It would have been better for the model to justify its calculation of I_2, even just by, e.g., setting up the integral and showing what it simplifies to after the substitution."*
- *"This Response is correct. While there are some structural/notational differences (e.g. the use of colors and XOR notation), the underlying fundamental argument is the same as the reference proof. They both use the strategy of dividing chessboard squares into classes based on row/column parity and use these classes to identify a winning strategy for Alice based on the allowed piece movements."*

Examples of weak correctness assessments:

- *"Flase, actual false."*
- *"Wrong claim that the statement is true."*
- *"Response A is proving a false statement. It claims we can arrange something without justification."*

How to fill out the "Preference Justification"

Preference Justification

Explain why you chose one proof over the other (2-3 sentences).

Please write 2–3 sentences that indicate whether Response A and Response B are correct or incorrect, and explain why you prefer one proof over the other by comparing and contrasting their approaches, styles, or methods.

There are many different factors you can use to determine which of the two responses is stronger.

- Strong proofs will be correct and clear, and should use few prerequisites, be fully rigorous, justify every logical step, and ultimately help the reader better understand the statement.
- Weak proofs may be wrong, contain incorrect statements, be confusing, skip justification, lean unnecessarily on advanced tools when simpler methods would suffice, or leave the statement no clearer than before.

Even if you think the two proofs are extremely similar, you should still try to find a way to differentiate them and pick one that you prefer, even if it is a very slight preference.

- For example: does one model restate the problem or summarize the result more nicely? Is one proof slightly more concise?

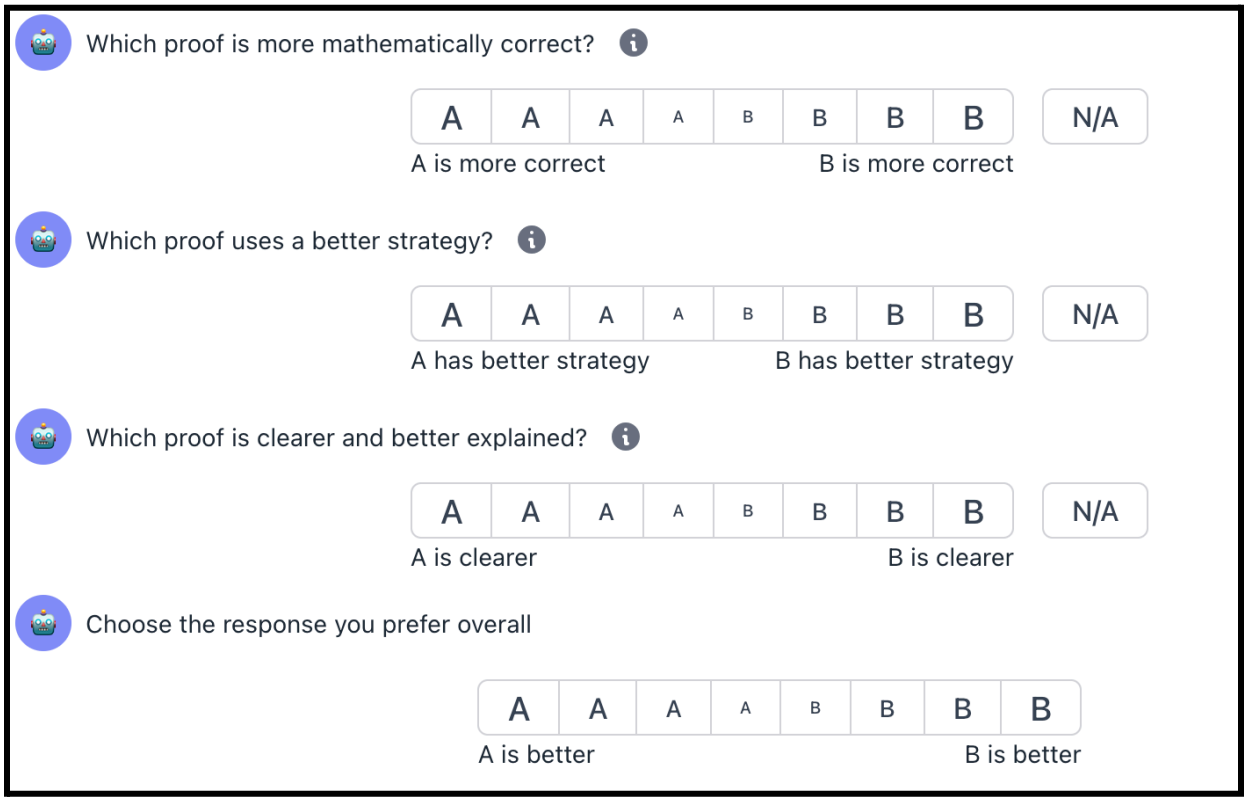
Examples of strong preference justifications:

- "*Response A is trying to apply nerve theorem directly, which is a wrong approach. It can be fixed with a considerable effort. Response B's approach is much more effective on top of being the only one correct.*"
- **"***Slight preference towards A because A correctly determines that f(-1) is an integer, but does so in an incorrect manner. Note that both A and B make similar crucial mistakes."*
- *"While Response A provides a slightly more clean proof, it writes the proof with ϵ and R_0 unfixed, which is not best practice. It would have been better to start the proof by first choosing $\epsilon = \frac{A}2$, then choosing a specific R_0 based on this ϵ. Response B uses the same general technique as Response A, but is more careful and exact with its statements."*

Examples of weak preference justifications:

- *"Both proofs are bad."*
- *"Model A's second proof was less bad than Model B's."*

How to Rank Your Preferences



Please rank the two proofs according to the four questions given.

- The "Choose the response you prefer overall" ranking should align with your answer in the "Preference Justification" section.
- Try to avoid using the "N/A" answer unless the proofs are essentially identical.

Below are some examples of how you might think about ranking the two models for each question.

Which proof is more mathematically correct?

- *Ex: Is one proof correct and the other proof incorrect?*
- *Ex: If both models divide the proof into cases, does one of the models give correct proofs for more of the cases than the other model does?*
- *Ex: If both models give correct proofs overall, does one model make a minor calculation error that doesn't impact the overall answer?*

Which proof uses a better strategy?

- *Ex: Does one proof use a strategy that might help the reader better understand how to approach similar problems?*
- *Ex: Does one proof go through and check all cases, while the other proof (correctly) simplifies the problem and reduces it to checking a subset of all cases?*
- *Ex: Does one proof use complicated or "high-powered" machinery to solve the problem, when a simpler approach would have sufficed?*

Which proof is clearer and better explained?

- *Ex: Is one proof more concise (while still providing sufficient justification for all claims)?*
- *Ex: Does one proof provide insufficient justification for part of its argument?*
- *Ex: Does one proof give context for the major theorem it uses so that the reader would be able to look it up, while the other one just refers to it as "a key theorem"?*
- *Ex: Does one proof restate the problem or summarize the conclusion in a nice way that is helpful to the reader?*

Choose the response you prefer overall

- Your answer to this question should align with your written answer in the "Preference Justification" section.
- See the section "How to fill out the 'Preference Justification'" above, for advice on how to compare the two models.

🔮 2.3 Reviewer Pod Resources

This is a living document. Last Updated 11/5/25

Welcome to the Project Phoenix Reviewing Team! This tab outlines how our review team operates. Here we detail the standards, expectations, and resources for you as a reviewer on project phoenix.

Onboarding + New Reviewer Actions

New to the project and reviewing (new specialists or reviewers transferred from other projects)

1. Complete task as fellow task is approved
2. Complete first review but do not submit. Send to pod Slack channel for review feedback.
3. Feedback on review with pod leader
4. First review approved by other reviewer! You have access to unlimited reviews

Was a fellow, now promoted to a reviewer

1. Complete first review but do not submit. Send to pod Slack channel for review feedback.
2. Feedback on review with pod leader
3. First review approved by other reviewer! You have access to unlimited reviews

Expectations (once onboarding + Actions above are complete)

- Complete at least 20 reviews per week (both rejections and approvals count toward total reviews, ~15 hours total)
- Maintain consistent quality and timeliness in reviews. See **2.1 How to evaluate fellow tasks in Platform 1** 🟩 and **2.2 How to evaluate model responses in Platform 2** 🟫 for quality guidance.
- You may only have 3 tasks claimed at a time and one timer running at a time.
- Complete one 1:1 meeting with your pod leader per week to check in. (Book 1:1s with your pod leader below!)
- Communicate proactively with your pod leader if you can't meet weekly goals.

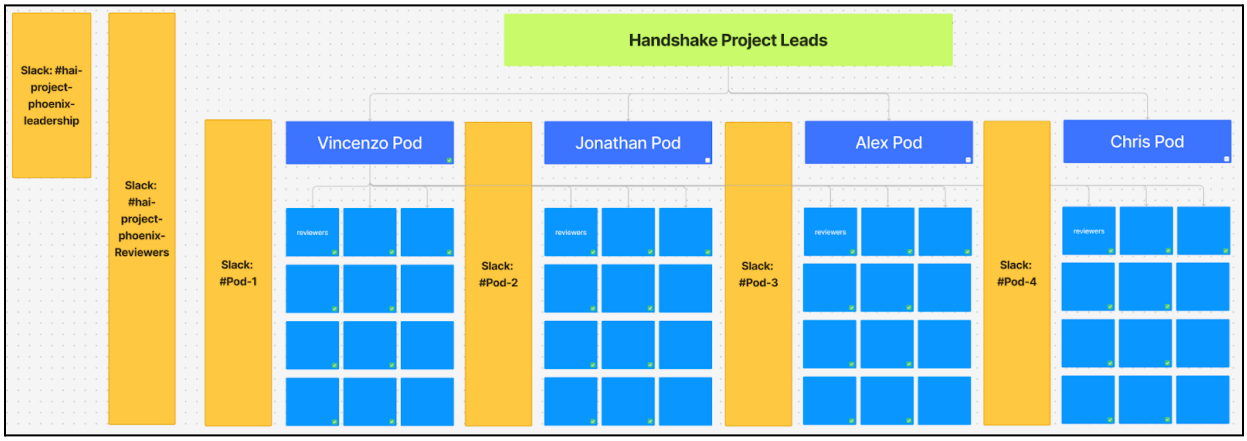
Resources:

- **Pod Tracker**

    - **See your total reviews and your availability response for this week**
    - **See if you need to schedule a 1:1 with your Pod lead!**
- [Audit sheet](#) (for pod leads to check their reviewer's recent work only)
- Schedule a meeting with **Vincenzo** [Click me!]
- Schedule a meeting with **Chris** [Click me!]
- Schedule a meeting with **Jonathan** [Click me!]
- Schedule a meeting with **Nasser** [Click me!]
- Schedule a meeting with **Mohammad** [Click me!]
- Schedule a meeting with **Finn** [Click me!]

Project Phoenix Reviewers Are Paired into Teams during onboarding - each team is managed by one extremely experienced Phoenix reviewer.

- Each Pod has approximately 10-15 reviewers
- Each Pod has its own Slack channel for specific questions and discussions
- Each Pod is a team!
- Each pod leader will meet with **each reviewer at least once per week (schedule meetings using the links above!)**
    - **The reviewers are responsible for scheduling a meeting, not the pod leader.**
- **Pod leaders will help with reviewing coaching, accountability, and are your first line of communication. You can also always book a meeting with Finn, your Handshake Project Lead**
- See the visualization below:



🏆 2.4 Golden Review Examples

This is a living document. Last Updated 11/5/25
[unfinished and in progress!]

Click any link below to watch video walkthroughs of great Reviews! These tutorials are for Reviewers to learn how to review fast and effectively.

Analysis
- Real Analysis
- Complex Analysis
- Functional Analysis
- Measure Theory
- Ordinary Differential Equations (ODEs)
- Partial Differential Equations (PDEs)

Algebra
- Group Theory
- Commutative Algebra
- Noncommutative Algebra
- Homological Algebra

Number Theory
- Algebraic Number Theory
- Analytic Number Theory
- Combinatronics

Geometry
- Algebraic Geometry
- Differential Geometry

Topology
- General (Point-Set) Topology
- Algebraic Topology
- Differential Topology
- Low-Dimensional Topology

Logic & Foundations
- Set Theory

- Mathematical Logic
- Category Theory

Probability & Statistics

? 2.5 Reviewer FAQs

This is a living document. Last Updated 11/5/25
[unfinished and in progress!]

Onboarding

P1

P2

Misc